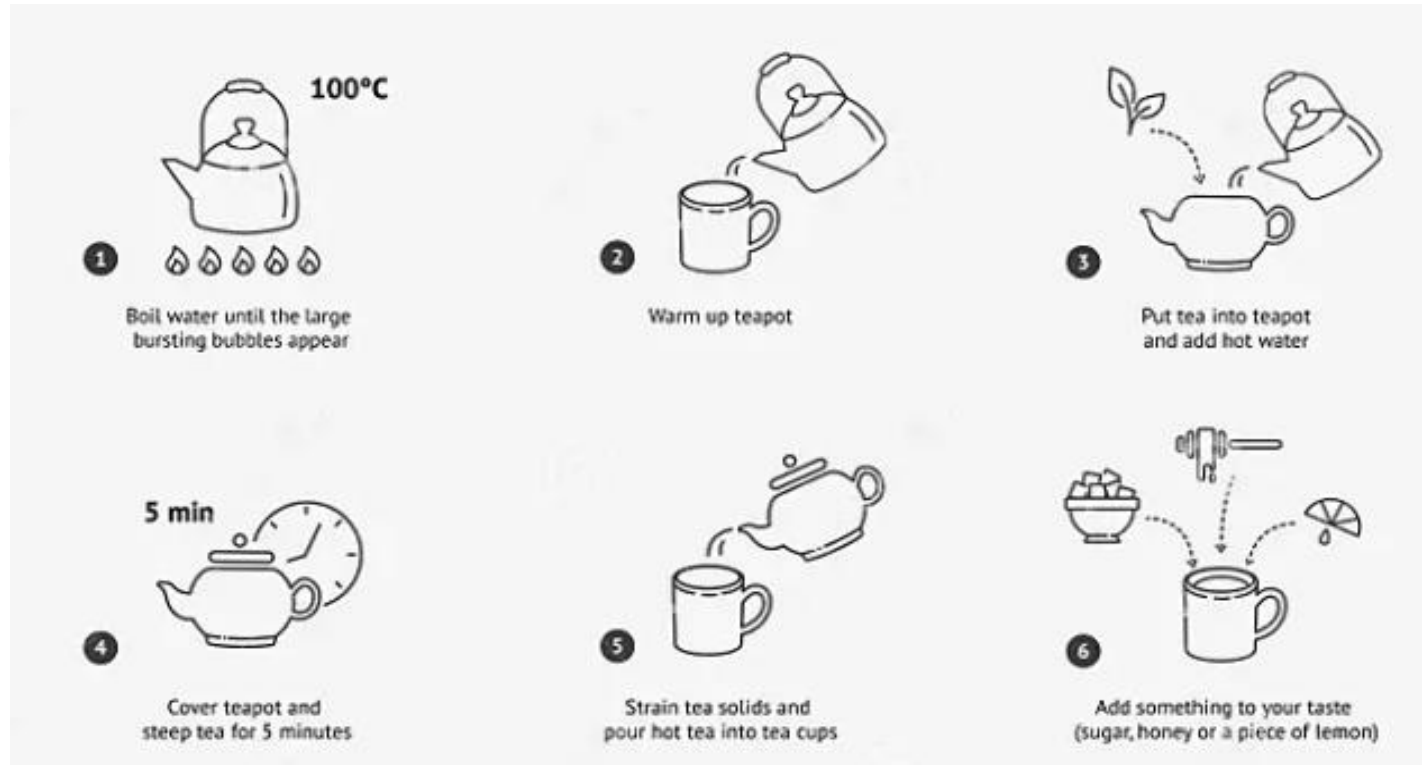


Introduction to types of Algorithms

-Ajay Iyengar

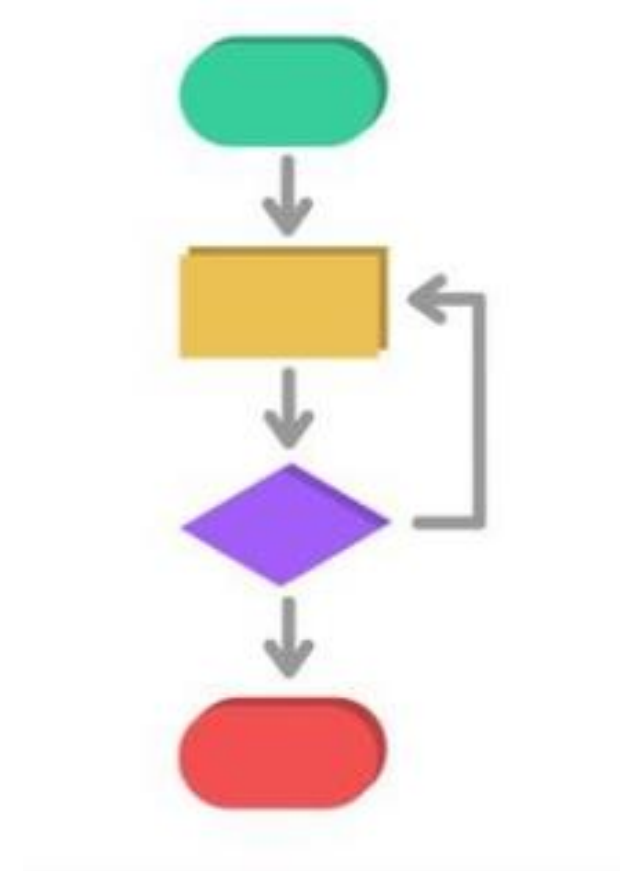
Email: iyengarajay@gmail.com

How to make Tea ?



What is an Algorithm ?

- An algorithm is a step-by-step procedure or formula for solving a problem
- Making Tea, Recipe for a cake, Sorting a list, or finding the shortest path
- Why study Algorithms ?
 - To solve problems efficiently
 - To understand the trade-offs between resources (time and space)



What is an Operation ?

- An ****operation**** is a basic instruction that takes constant time to execute.
- Examples of operation include
 - **Arithmetic:** $a + b$, $x * y$, $i++$
 - **Comparison:** $a < b$, $\text{arr}[i] == \text{target}$
 - **Assignment:** $\text{sum} = 0$, $\text{arr}[i] = 5$
 - **Array access:** $\text{arr}[i]$, $\text{matrix}[i][j]$
 - **Return statement:** `return result;`

These are simple, elementary steps. More complex operations like `Arrays.sort()` or `list.contains()` are made up of many such basic operations.

Analyzing Algorithms

- Imagine you've written two programs that solve the same problem:
 - **Program A:**
 - 100 inputs $\rightarrow$ 100 operations
 - 200 inputs $\rightarrow$ 200 operations (2x input = 2x operations)
 - **Program B:**
 - 100 inputs $\rightarrow$ 10,000 operations
 - 200 inputs $\rightarrow$ 40,000 operations (2x input = 4x operations!)
- **With 1 million inputs:**
 - Program A: 1,000,000 operations
 - Program B: 1,000,000,000,000 operations (1 trillion!)

Time & Space Complexity : Importance

- To evaluate the efficiency of an algorithm
- Why Time complexity ?
 - Helps in comparing algorithms for large data sets(Which solution is better ?)
 - Crucial for applications where performance at scale matters
- Why Space complexity ?
 - Measures how much memory an algorithm uses
 - Crucial for applications with limited memory resources

Asymptotic Analysis

- Also called asymptotics – Method of describing limiting behavior
- Properties of a function $f(n)$, as n becomes large
- Suppose: $f(n) = n^2 + 3n$
 - As n becomes very large, $3n$ becomes insignificant as compared to n^2
- The function $f(n)$ is said to be "*asymptotically equivalent* to n^2 , as $n \rightarrow \infty$

Example 1 : Asymptotic Analysis

```
public void exampleAlgorithm1(int n) {  
    // Constant operations: 10 operations  
    for (int i = 0; i < 10; i++) {  
        System.out.println("Constant operation " + i);  
    }  
  
    // Linear operations: 3n operations  
    for (int i = 0; i < n; i++) {  
        System.out.println("Operation 1 for input " + i); // n times  
    }  
    for (int i = 0; i < n; i++) {  
        System.out.println("Operation 2 for input " + i); // n times  
    }  
    for (int i = 0; i < n; i++) {  
        System.out.println("Operation 3 for input " + i); // n times  
    }  
}
```

Total complexity: $T(n) = 3n + 10$

Constants are irrelevant as they don't scale with 'n'

Total complexity: $T(n) = 3n$

Coefficients (e.g., 333) are ignored because they don't affect the **rate of growth**—only the **magnitude**

Total complexity: $T(n) = n$

Asymptotic analysis captures how the algorithm scales with n, not its absolute runtime.

Example 2 : Asymptotic Analysis

```
public void exampleAlgorithm2(int n) {  
    // Constant operations: 3 operations  
    int a = 0, b = 0, c = 0; // 3 operations  
    // Linear operations: 2n  
    for (int i = 0; i < n; i++) {  
        System.out.println("Linear operation " + i); // n times  
    }  
    for (int i = 0; i < n; i++) {  
        System.out.println("Another linear operation " + i); // n times  
    }  
    // Quadratic operations: n2  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            System.out.println("Quadratic operation for (" + i + ", " + j + ")"); // n2 times  
        }  
    }  
}
```

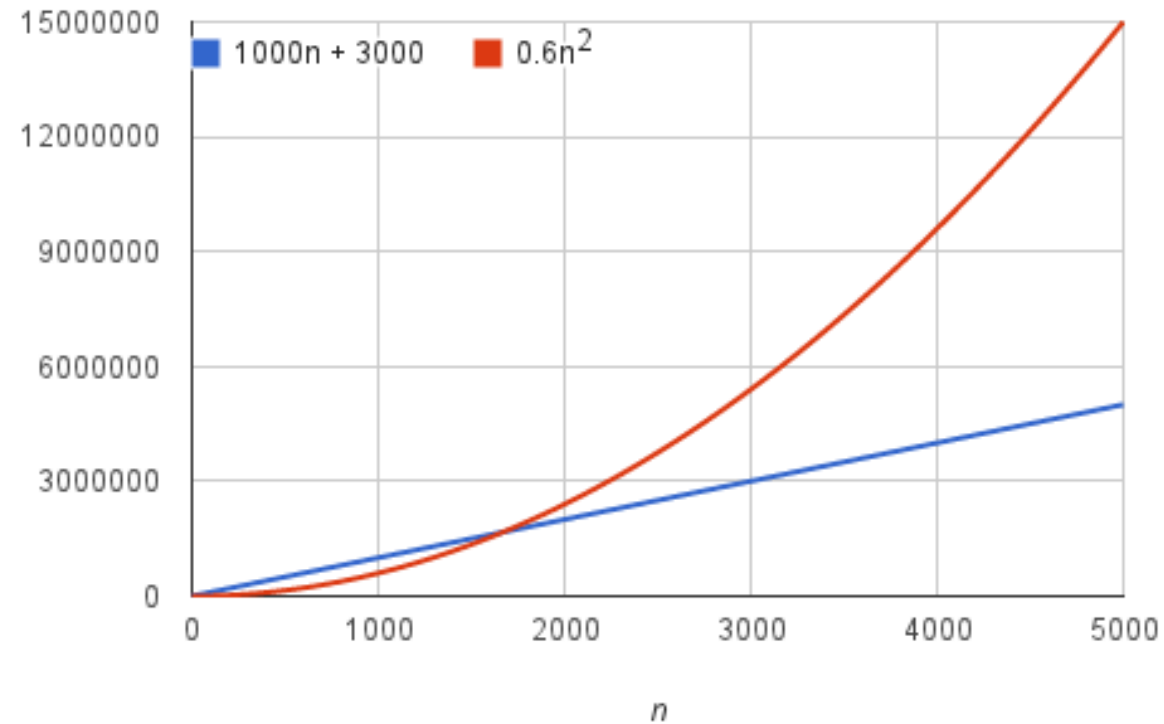
Total complexity: $T(n) = n^2 + 2n + 3$

Total complexity: $T(n) = n^2 + 2n$

Total complexity: $T(n) = n^2$

Asymptotic Notation : Conclusion

- $0.6n^2$ & $1000n+3000$



- Drop the less significant terms and constant co-efficients & focus on rate of growth

3 forms of Asymptotic Notation

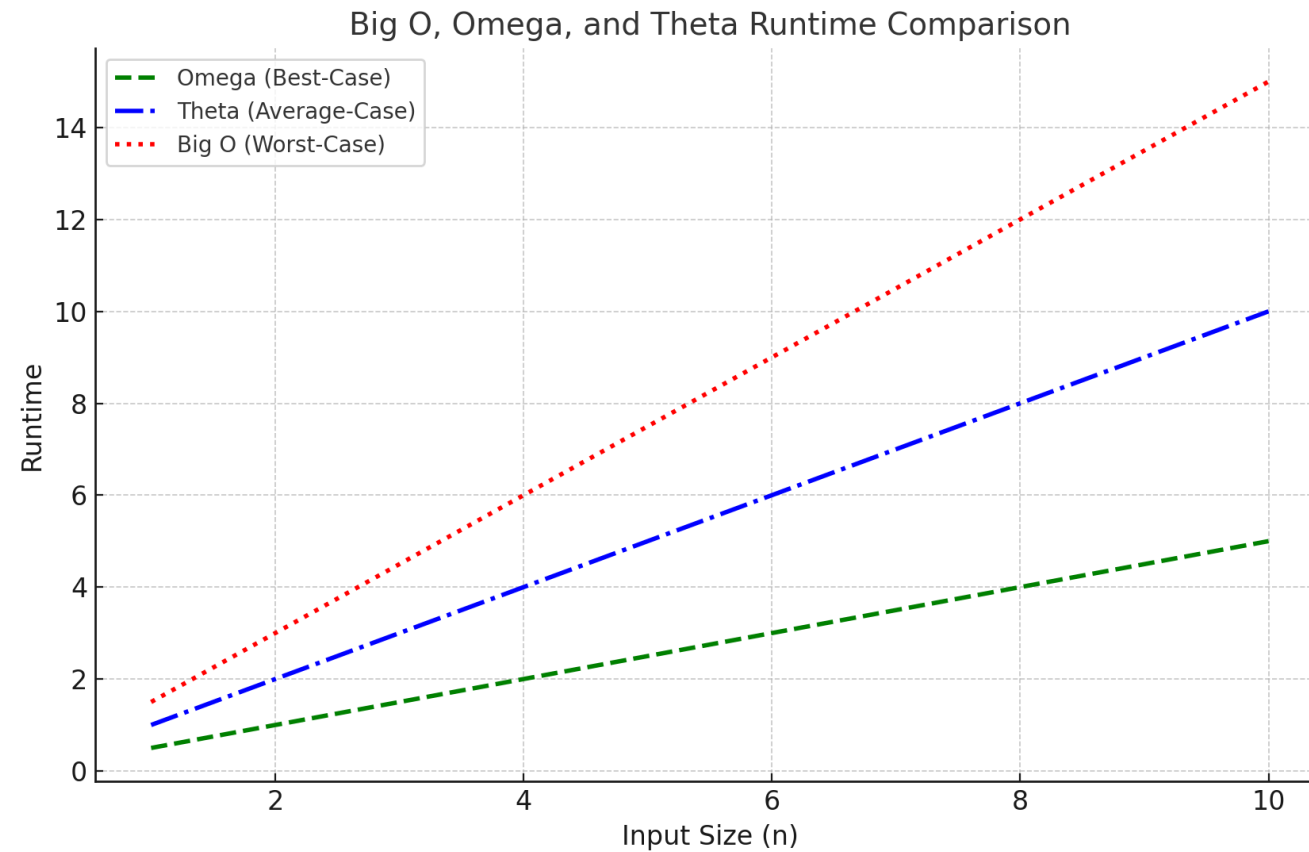
- **Big O (Worst-Case Analysis)**
 - Describes the upper bound on time/space requirements
 - Answers : “What is the longest time this algorithm will take ?”
- **Big- Ω -Big-Omega (Best-Case Analysis)**
 - Describes the lower bound on time/space requirements
 - Answers : “What is the shortest time this algorithm can take ?”
- **Big- θ - Big-Theta(Average-Case Analysis)**
 - Describes the exact bound for typical input
 - Answers : “What is the expected runtime ?”

Introduction to Big O notation

“The language of complexity”

- **What is Big O?**
 - A mathematical notation to describe the efficiency of algorithms.
 - Focuses on the growth rate of an algorithm's time or space requirements relative to input size (n).
- **Why is it important?**
 - To analyze and compare algorithms.
 - To predict performance for large inputs.

Big O vs. Omega vs. Theta



Big O notation : Key concepts

- Time Complexity
 - refers to the **amount of computer time** an algorithm takes to complete as a function of the size of the input
 - Usually expressed in Big O notation (e.g., $O(n)$, $O(n^2)$, $O(\log n)$)
- Space Complexity
 - refers to the **amount of (extra) memory** an algorithm needs to run as a function of the size of the input.
 - Usually expressed in Big O notation (e.g., $O(n)$, $O(n \log n)$)

Space Complexity deals with memory usage, while **Time Complexity** deals with the number of operations*

Time Complexity

What it is and What it isn't...

- Not concerned with “actual time” it takes for algorithm to run on the machine
- It is about how the runtime increases as input size grows
- It is the rate or growth pattern of *computer time* taken relative to how large the input is
- **It is not about exact duration**

Computer time deals the number of basic operations (additions, comparisons, assignments, etc.) the algorithm performs, NOT actual seconds or milliseconds on your machine.

Time Complexity : Growth pattern

- **Algorithm A : $O(n)$:** This algorithm takes a certain amount of time to process each element in the input list, and the time taken increases linearly with the size of the input.
- **Algorithm B : $O(n^2)$:** This algorithm's runtime increases quadratically with the size of the input
- Let's assume that:
 - For **Algorithm A**, it takes 1 unit of work to process 1 element, 2 units of work to process 2 elements, 3 units of work for 3 elements, and so on – **Linear**
 - For **Algorithm B**, it takes 1 unit of work to process 1 element, 4 for 2 elements, 9 units of work for 3 elements, and so on, 16 for 4 and so on - **Quadratic**

Time Complexity : Growth pattern continued...

- Run algorithm A and B on same input size
 - Concerned about how runtime grows when input size increases
 - Not concerned about actual time they take to process each element
- Algorithm A : $O(n)$: If you double the input size, the runtime doubles.
- Algorithm B : $O(n^2)$: If you double the input size, the runtime quadruples.

Examples of Time Complexity

Time Complexity	Meaning	Description	Example
$O(1)$	Constant Time	Runtime does not depend on input size	Accessing a specific element in an array by index, such as <code>arr[i]</code> .
$O(\log n)$	Logarithmic Time	The runtime grows logarithmically with the input size.	Binary search in a sorted array. Each step halves the search space.
$O(n)$	Linear Time	The runtime grows linearly with the input size.	Looping through an array or list to find an element or sum the values.
$O(n^2)$	Quadratic Time	The runtime grows quadratically with the input size.	Nested loops over a list (e.g., bubble sort or selection sort).

More examples of Time Complexity

Time Complexity	Meaning	Description	Example
$O(n \log n)$	Linearithmic Time	The runtime grows faster than linear but slower than quadratic.	Efficient sorting algorithms like Merge Sort or Quick Sort.
$O(n^3)$	Cubic Time	The runtime grows cubically with the input size.	Triple nested loops, such as finding all triplets in an array that sum to a target value.
$O(n!)$	Factorial Time	The runtime grows extremely fast as the input size increases.	Generating all permutations of a set of elements (e.g., brute force for combinatorics).

Complexity Hierarchy(Fastest to Slowest)

- $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$

[BEST -----> WORST]

- **Visual Growth for $n = 100$:**

- $O(1) \rightarrow 1$ operation (constant)
- $O(\log n) \rightarrow \sim 7$ operations
- $O(n) \rightarrow 100$ operations
- $O(n \log n) \rightarrow \sim 700$ operations
- $O(n^2) \rightarrow 10,000$ operations
- $O(2^n) \rightarrow 1,267,650,600,228,229,401,496,703,205,376$ operations ☠

Deriving Big O : $O(1)$ | Constant Time

No matter how big the input, always takes the same time

```
// Example 1: Accessing array element
public int getValue(int[] arr) {
    return arr[0]; // One operation, always
}
```

```
// Example 2: Simple arithmetic
public int add(int a, int b) {
    int sum = a + b;
    return sum;
}
```

```
// Example 3: Multiple constant operations
public int calculate(int[] arr) {
    int x = arr[0]; // O(1)
    int y = arr[5]; // O(1)
    int z = x + y; // O(1)
    return z * 2; // O(1)
}
// Total: O(1) + O(1) + O(1) + O(1) = O(1) still!
```

Key : No loops, just direct access = $O(1)$

Deriving Big O : $O(n)$ | Linear Time

```
int sum = 0;           // O(1)
for (int i = 0; i < n; i++) { // O(n)
    sum += i;           // O(1) per iteration
}
System.out.println(sum); // O(1)
```

- **Total Complexity:**
 - Initialization: $O(1)$
 - Loop: $O(n)$ iterations $\times O(1)$ work = $O(n)$
 - Print statement: $O(1)$
 - **Overall:** $O(n)$

Deriving Big O : $O(n)$ | Linear Time

Runtime grows directly with input size, Double input = Double Runtime

```
// Example 1: Print all elements
public void printArray(int[] arr) {
    for (int i = 0; i < arr.length; i++) { // Runs n times
        System.out.println(arr[i]);
    }
}
```

```
// Example 2: Find maximum element
public int findMax(int[] arr) {
    int max = arr[0];
    for (int i = 1; i < arr.length; i++) { // n-1 times ≈ n times
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}
```

Deriving Big O : $O(n)$ | Linear Time

Runtime grows directly with input size, Double input = Double Runtime

```
// Example 3: Multiple separate loops
public void process(int[] arr) {
    // First loop
    for (int i = 0; i < arr.length; i++) { // n times
        System.out.println(arr[i]);
    }

    // Second loop
    for (int i = 0; i < arr.length; i++) { // n times
        System.out.println(arr[i] * 2);
    }
}

// Total:  $O(n) + O(n) = O(2n) = O(n)$  [We drop constants!]
```

Key : One loop iterating through input = $O(n)$

Deriving Big O : $O(n^2)$ | Quadratic Time

Runtime grows with square of input. Double input = 4x time!

```
for (int i = 0; i < n; i++) {           // Outer loop:  $O(n)$ 
    for (int j = 0; j < n; j++) {       // Inner loop:  $O(n)$ 
        System.out.println("Hello");   //  $O(1)$  per iteration
    }
}
```

Total Complexity:

- Outer loop: Runs n times.
- Inner loop: Runs n times for each iteration of the outer loop.
- Total complexity: $n \times n = O(n^2)$

Log & Exponential

- $\log_{10}(1000) = 3$ since $1000 = 10^3$
- $\log_2 16 = 4$, since $2^4 = 2 \times 2 \times 2 \times 2 = 16$.
- The binary logarithm uses base 2 and is frequently used in computer science

Concept	Meaning	Example
Exponent	Multiply repeatedly	$2^4 = 16$
Logarithm	“How many times did I multiply ?”	$\log_2(16) = 4$

Deriving Big O : $O(\log n)$

Total Complexity: $O(\log n)$

- Starts with 1
- Loop runs as long as $i \leq n$
- At each iteration $i = i * 2$
 - Values of i are 1, 2, 4, 8, 16...
- After 'k' iterations, $i = 2^k$
- Loop ends when $2^k > n$
- Take log base 2, $k = \log_2 n$
- Loop runs $\sim \log_2(n)$ times
- Each iteration takes $O(1)$

```
for (int i = 1; i <= n; i = i * 2) {  
    System.out.println("Hello");  
}
```

Deriving Big O : $O(\log n)$

```
for (int i = 16; i > 0; i = i / 2) {  
    System.out.println(i);  
}
```

****Iterations:****

Iteration 1: $i = 16 \rightarrow \text{print } 16 \rightarrow i \text{ becomes } 8$
Iteration 2: $i = 8 \rightarrow \text{print } 8 \rightarrow i \text{ becomes } 4$
Iteration 3: $i = 4 \rightarrow \text{print } 4 \rightarrow i \text{ becomes } 2$
Iteration 4: $i = 2 \rightarrow \text{print } 2 \rightarrow i \text{ becomes } 1$
Iteration 5: $i = 1 \rightarrow \text{print } 1 \rightarrow i \text{ becomes } 0$
(loop stops: $i = 0$, condition $i > 0$ is false)

Iteration 1 $\rightarrow n / 2$

Iteration 2 $\rightarrow n / 4$

Iteration 3 $\rightarrow n / 8$

Iteration $k \rightarrow n / 2^k$

1. We stop when n reaches 1:

$$n / 2^k = 1$$

2. Multiply both sides by 2^k :

$$n = 2^k$$

3. Take log base 2 of both sides:

$$\log_2(n) = k$$

Common Misconception

- Is this $O(\log n)$?

```
for (int i = n; i > 0; i = i - n/2) { // SUBTRACTING n/2
    System.out.println(i);
}
```

- We start from $i=n$
- In every iteration we subtract $n/2$ (constant amount relative to n)
- Example : if $n = 100$, then
 - $N/2 = 50$
 - i becomes: $100 \rightarrow 50 \rightarrow 0 \rightarrow \text{stop}$
 - Only 2 iterations

Time Complexity : $O(1)$,does not grow with n , executes constant number of times

Quick Test

A. `for (int i = n; i > 1; i = i / 2) { }`

✓ $O(\log n)$ - dividing by 2

B. `for (int i = 1; i <= n; i = i * 3) { }`

✓ $O(\log n)$ - multiplying by 3

C. `for (int i = n; i > 0; i = i - 1) { }`

✓ $O(n)$ - decrementing by 1

D. `for (int i = 0; i < n; i = i + n/2) { }`

✓ $O(1)$ - only 2 iterations regardless of n

E. `for (int i = n; i > 1; i = i / 10) { }`

✓ $O(\log n)$ - dividing by 10

$O(\log n)$ – Quick Rule

- If your loop variable is MULTIPLIED or DIVIDED by a constant each iteration, it's $O(\log n)$
- Examples:
 - $i = i * 2 \rightarrow O(\log n)$
 - $i = i / 3 \rightarrow O(\log n)$
 - $i = i * 5 \rightarrow O(\log n)$
 - $i = i / 10 \rightarrow O(\log n)$
- But:
 - $i = i + 2 \rightarrow O(n)$
 - $i = i - 5 \rightarrow O(n)$

Deriving Big O : $O(n^2)$

Total Complexity: $O(n^2)$

- Line 1 : $O(1)$
- Line 2 : Outer Loop : 'n' iterations
- Line 3/4 : Inner Loop : Depends on 'i'
 - $i = 0$: 0 times, $i = 1$: 1 time, $i = 2$: 2 times, $i = n-1$: $n - 1$ times
 - Total work = $0+1+2+3+\dots+(n-1)$ (Sum of first natural $n-1$ numbers)
$$= n(n-1)/2 = n^2/2 = n^2$$
- Dropping constants : $O(n^2)$

```
int sum = 0; // Line 1
for (int i = 0; i < n; i++) { // Line 2
    for (int j = 0; j < i; j++) { // Line 3
        sum += i; // Line 4
    }
}
```

Tip : Sum of first 'k' natural numbers is $k(k+1)/2$. Replace k with $n-1$

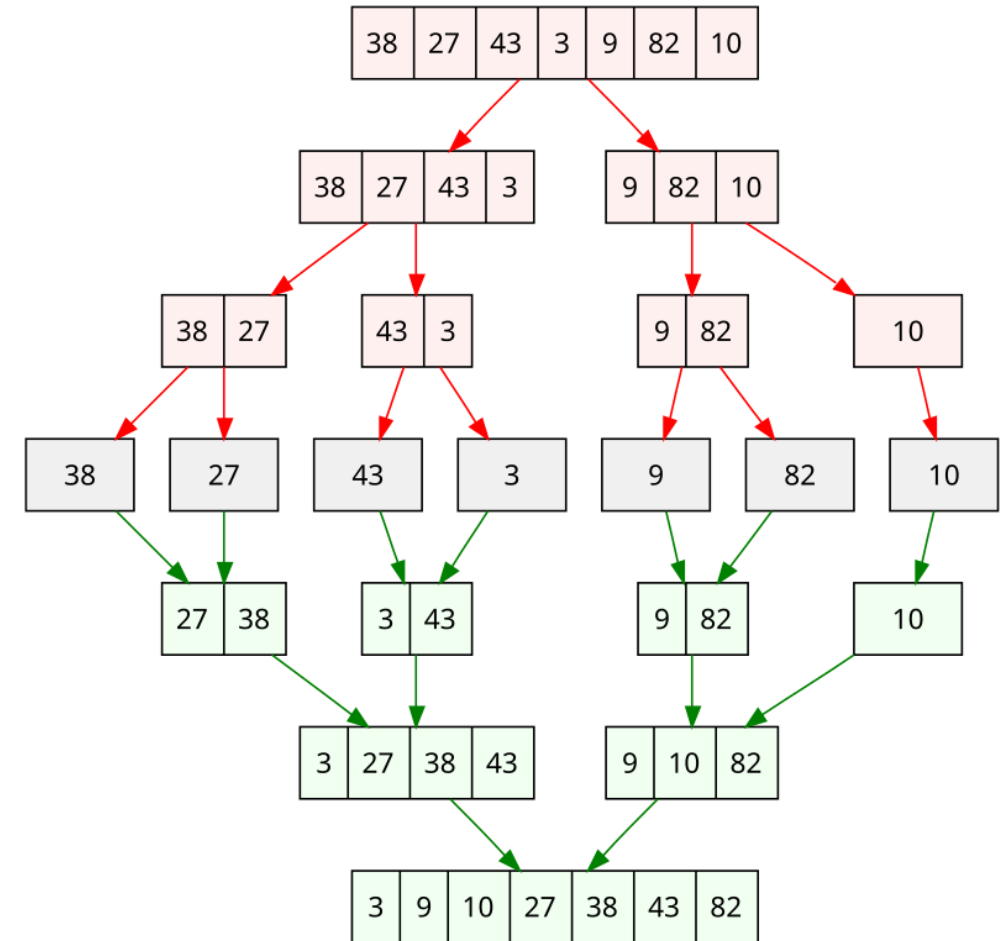
Binary Search : $O(\log n)$?

- Halves the search space in each step
 - After the first comparison: $n \rightarrow n/2$
 - After the second comparison: $n/2 \rightarrow n/4$.
 - After k comparisons: $n \rightarrow n / 2^k$
- Search stops when $n/2^k = 1$ where k is the number of comparisons
- $n/2^k = 1$ or $n = 2^k$
- Take base 2 logarithm on both side : $\log_2(n) = \log_2(2^k)$ i.e. $k * \log_2(2)$
- $\log_2(n) = k$ i.e. $k = \log(n)$

Deriving Big O : $O(n \log n)$

Total Complexity : $O(n \log n)$

- **Recursive Levels:**
 - Each division reduces the size of sub arrays by half.
 - Total levels = $\log_2(n)$
- **Work Per Level:**
 - Merging involves $O(n)$ at every level because all elements are processed.
- **Total Work:**
 - $O(n)$ work per level * $\log_2(n)$ levels
= $O(n \log n)$

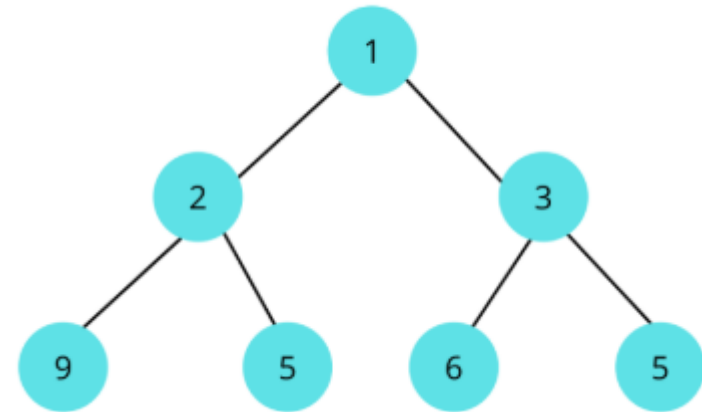


BST : Time complexities

- Traversal - in-order, pre-order, post-order : $O(n)$
- Searching, Inserting & Deleting a node
 - Balanced : $O(\log n)$
 - Unbalanced(skewed) : $O(n)$
- Find minimum/maximum element or Height of a tree
 - Balanced : $O(\log n)$
 - Unbalanced(skewed) : $O(n)$

Why is the height of a balanced tree $O(\log n)$?

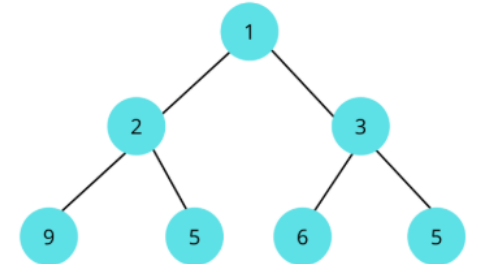
- In a balanced BST, each node has at most 2 children
- Height of a tree is defined as number of edges on the longest path from the root to a leaf node
- Why logarithmic height ?
 - Level 0 : 1 node = 2^0
 - Level 1 : Up to 2 nodes = 2^1
 - Level 2 : Up to 4 nodes = 2^2
 - Level 3 : Up to 8 nodes = 2^3
 - At each level, it doubles. At Level 'h' = 2^h



Why is the height of a balanced tree $O(\log n)$?

- The total number of nodes N in the tree with height h is the sum of all nodes at each level: $N = 2^0 + 2^1 + 2^2 + \dots + 2^h$
- **Geometric series** where the first term is 2^0 , the common ratio is 2, and the number of terms is $h+1$ (since the tree has $h+1$ levels, from level 0 to level h).
- The sum S of the first 'n' terms of this series is:
$$S = \frac{a(r^n - 1)}{r - 1}$$
 - a is the first term ($2^0 = 1$)
 - r is the common ratio and (2)
 - n is the number of terms ($h+1$, we have $h+1$ levels)
- Total nodes in a balanced tree of height 'h' is

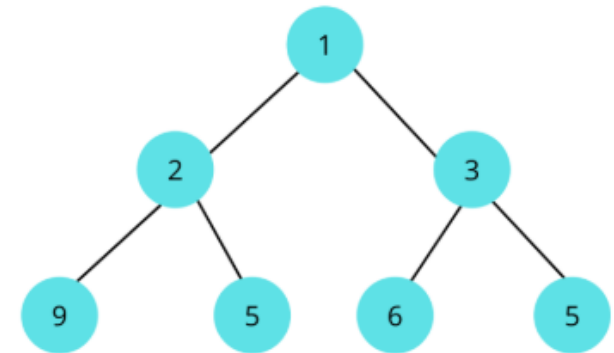
$$N = \frac{1 \times (2^{h+1} - 1)}{2 - 1} = 2^{h+1} - 1$$



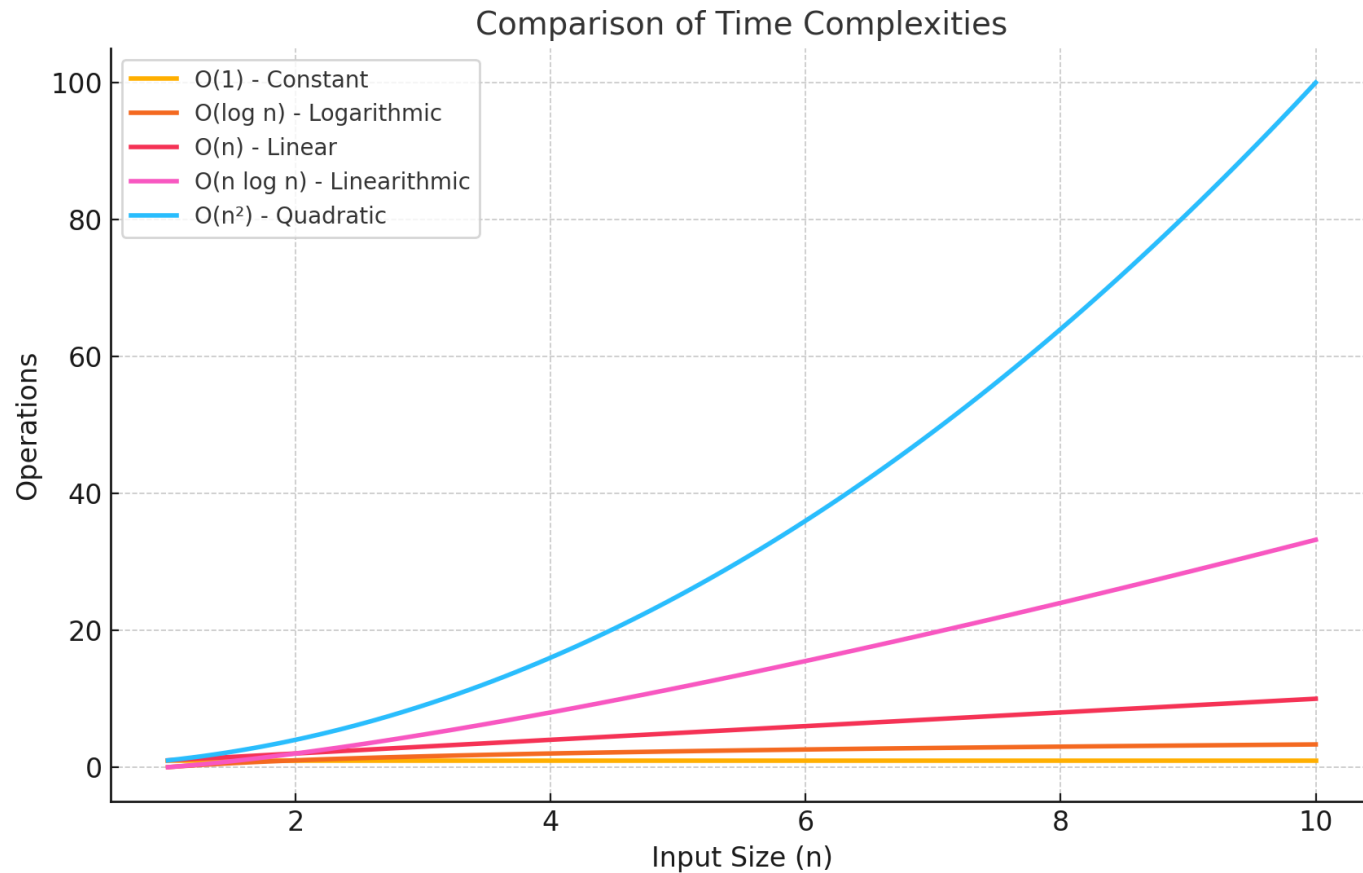
Why is the height of a balanced tree $O(\log n)$?

- Total nodes 'N' in a balanced tree of height 'h' is
- $N = 2^{h+1} - 1$ or $N \approx 2^{h+1}$ or
- $h + 1 \approx \log n$
- Height of a balanced tree is $O(\log n)$

$$N = \frac{1 \times (2^{h+1} - 1)}{2 - 1} = 2^{h+1} - 1$$



Comparison of Time complexities



Time Complexity : Tricky Scenarios

- Scenario 1 : Dependent Loops

```
// Are these nested?  
for (int i = 0; i < n; i++) {  
    for (int j = i; j < n; j++) { // Starts from i, not 0!  
        System.out.println(i + ", " + j);  
    }  
}
```

Analysis:

When $i=0$, inner runs n times

When $i=1$, inner runs $n-1$ times

When $i=n-1$, inner runs 1 time

Total: $n + (n-1) + (n-2) + \dots + 1 = n(n+1)/2 = \mathbf{O(n^2)}$

Pro Tip: Even if inner loop doesn't always run n times, if it's nested and depends on outer loop variable, usually $\mathbf{O(n^2)}$

Time Complexity : Tricky Scenarios

- Scenario 2 : Two different sized inputs

```
public void process(int[] arr1, int[] arr2) {  
    for (int i = 0; i < arr1.length; i++) {  
        for (int j = 0; j < arr2.length; j++) {  
            System.out.println(arr1[i] + arr2[j]);  
        }  
    }  
}
```

Time Complexity: $O(n \times m)$ where
 $n = \text{arr1.length}$, $m = \text{arr2.length}$

Time Complexity : Tricky Scenarios

- Scenario 3 : Early Termination

```
public boolean hasZero(int[] arr) {  
    for (int i = 0; i < arr.length; i++) {  
        if (arr[i] == 0) {  
            return true; // Might return immediately!  
        }  
    }  
    return false;  
}
```

Analysis:

Best case: $O(1)$ - zero at first position

Average case: $O(n/2) \approx O(n)$

Worst case: $O(n)$ - no zero, or zero at end

Time Complexity : Tricky Scenarios

- Scenario 4 : Multiple Loop Variables

```
public void weirdLoop(int n) {  
    for (int i = 1; i < n; i = i * 2) {           // Multiplies by 2  
        for (int j = 0; j < n; j++) {           // Goes to n  
            System.out.println(i + "," + j);  
        }  
    }  
}
```

Analysis:

Outer loop: $\log n$ times (multiplying)

Inner loop: n times each iteration

Total: **$O(n \log n)$**

2D Array Time Complexity Examples

- Example 1 : Print all Elements

```
public void printMatrix(int[][] matrix) {  
    for (int i = 0; i < matrix.length; i++) {           // runs n times (rows)  
        for (int j = 0; j < matrix[i].length; j++) {     // runs m times (columns)  
            System.out.println(matrix[i][j]);  
        }  
    }  
}
```

Analysis:

Outer loop: runs n times (n = number of rows)

Inner loop: runs m times (m = number of columns)

Total operations: $n \times m$

Time Complexity: $O(n \times m)$ or $O(n^2)$ if it's a square matrix ($n = m$)

2D Array Time Complexity Examples

- Example 2 : Sum of diagonal elements

```
public int sumDiagonal(int[][] matrix) {  
    int sum = 0;  
    for (int i = 0; i < matrix.length; i++) {           // runs n times  
        sum += matrix[i][i];                             // accessing diagonal element  
    }  
    return sum;  
}  
// Assuming square matrix: n x n
```

Analysis:

Single loop through diagonal only

For $n \times n$ matrix, only n elements accessed

Time Complexity: $O(n)$

2D Array Time Complexity Examples

- Example 3 : Matrix transpose

```
public void transpose(int[][] matrix) {  
    // For square matrix n x n  
    for (int i = 0; i < matrix.length; i++) {  
        for (int j = i + 1; j < matrix.length; j++) { // Start from i+1, not 0  
            // Swap matrix[i][j] with matrix[j][i]  
            int temp = matrix[i][j];  
            matrix[i][j] = matrix[j][i];  
            matrix[j][i] = temp;  
        }  
    }  
}
```

Analysis:

Focus on the inner loop as it

Time Complexity : Golden Rules

- **### Rule 1: Count the Loops**

No loops $\rightarrow O(1)$, One loop $\rightarrow O(n)$

Nested loops (2 deep) $\rightarrow O(n^2)$, Nested loops (3 deep) $\rightarrow O(n^3)$

- **### Rule 2: Sequential Operations Add Up**

```
for (int i = 0; i < n; i++) { }    // O(n)
for (int i = 0; i < n; i++) { }    // O(n)
// Total: O(n) + O(n) = O(2n) = O(n)
```

Time Complexity : Golden Rules

- #### Rule 3: Nested Operations Multiply

```
for (int i = 0; i < n; i++) {           // O(n)
    for (int j = 0; j < n; j++) {       // O(n)
        System.out.println(i + j);
    }
}
```

- #### Rule 4: Drop Constants

```
// 3 separate loops
O(3n) → O(n)
O(5n2) → O(n2)
O(100) → O(1)
```

Time Complexity : Golden Rules

- #### Rule 5: Drop Lower Order Terms

```
O(n2 + n) → O(n2)      // n2 dominates  
O(n3 + n2 + n) → O(n3) // n3 dominates  
O(log n + n) → O(n)      // n dominates
```

- #### Rule 6: Different Variables Stay Separate

```
public void process(int[] arr1, int[] arr2) {  
    for (int i = 0; i < arr1.length; i++) { } // O(n)  
    for (int j = 0; j < arr2.length; j++) { } // O(m)  
}  
// Total: O(n + m), NOT O(n)!
```

Space Complexity

- Analyzing how much memory an algorithm uses relative to the input size
- Constant Space Complexity : $O(1)$

```
public void swap(int a, int b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}
```

- Analysis
 - int a, b : Input
 - int temp : Extra memory used

Linear Space Complexity : $O(n)$

- Space grows linearly with input size

```
public int[] createArrayCopy(int[] arr) {  
    int[] copy = new int[arr.length];  
    for (int i = 0; i < arr.length; i++) {  
        copy[i] = arr[i];  
    }  
    return copy;  
}
```

- Analysis
 - `int [] copy` : Proportional to `arr.length` ($O(n)$)
 - `int i` : $O(1)$

Quadratic Space Complexity : $O(n^2)$

- Space grows quadratically with input size

```
public int[][] createMatrix(int n) {  
    int[][] matrix = new int[n][n];  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            matrix[i][j] = i + j;  
        }  
    }  
    return matrix;  
}
```

- Analysis – Matrix grows quadratically with n
 - `int [] [] matrix` : Requires n^2 space
 - `int i, j` : $O(1)$

Recursive Function : $O(n)$

- Each recursive call adds a new frame to the call stack, consuming memory

```
public int factorial(int n) {  
    if (n == 0) return 1;  
    return n * factorial(n - 1);  
}
```

- Analysis – $O(\text{Recursion depth} * \text{Space per call})$
 - Stack frames – One for each call.
 - Depth of recursion – n
 - Memory per call = Size of arguments + Return address + Overhead (Roughly 24-32 bytes per call)
 - If $n = 5$, roughly $5 * 32 = 160$ bytes

Logarithmic Space Complexity : $O(\log n)$

- Space usage decreases exponentially with the problem size

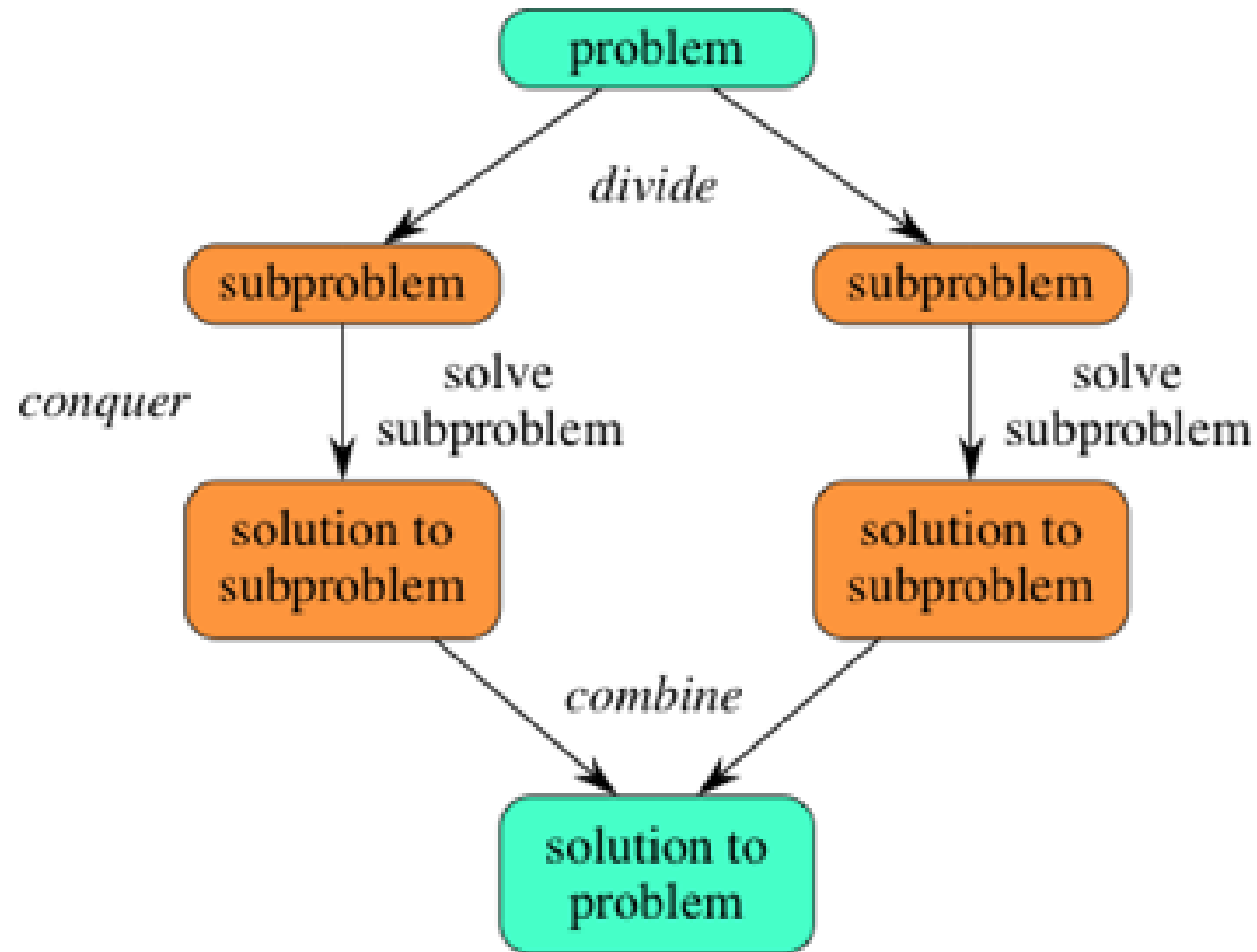
```
public int binarySearch(int[] arr, int target, int low, int high) {  
    if (low > high) return -1;  
    int mid = low + (high - low) / 2;  
    if (arr[mid] == target) return mid;  
    else if (arr[mid] > target) return binarySearch(arr, target, low, mid - 1);  
    else return binarySearch(arr, target, mid + 1, high);  
}
```

- Analysis –
 - Stack frames – One for each call.
 - Depth of recursion – $\log n$ (halving of array at each step)

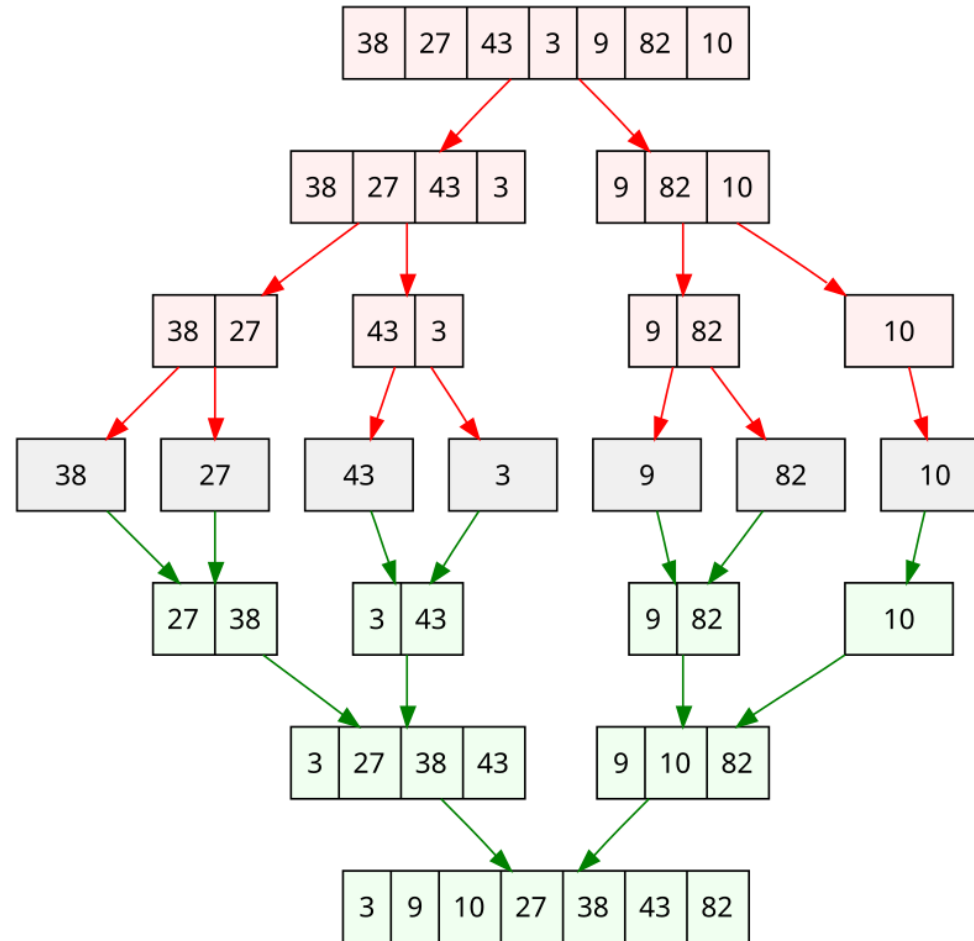
Divide-and-conquer algorithm

- At a high-level
 - Recursively breaks down a problem into 2 or more sub-problems(similar to original problem)
 - Solve each sub-problem
 - Solutions to sub-problems are then combined to give a solution to original problem
- Basis of algorithms for many problems, ex : sorting
 - Quick Sort
 - Merge Sort

Divide, Conquer, Combine



Divide-and-conquer : Merge Sort



$$O(n) \times O(\log n) = O(n \log n)$$

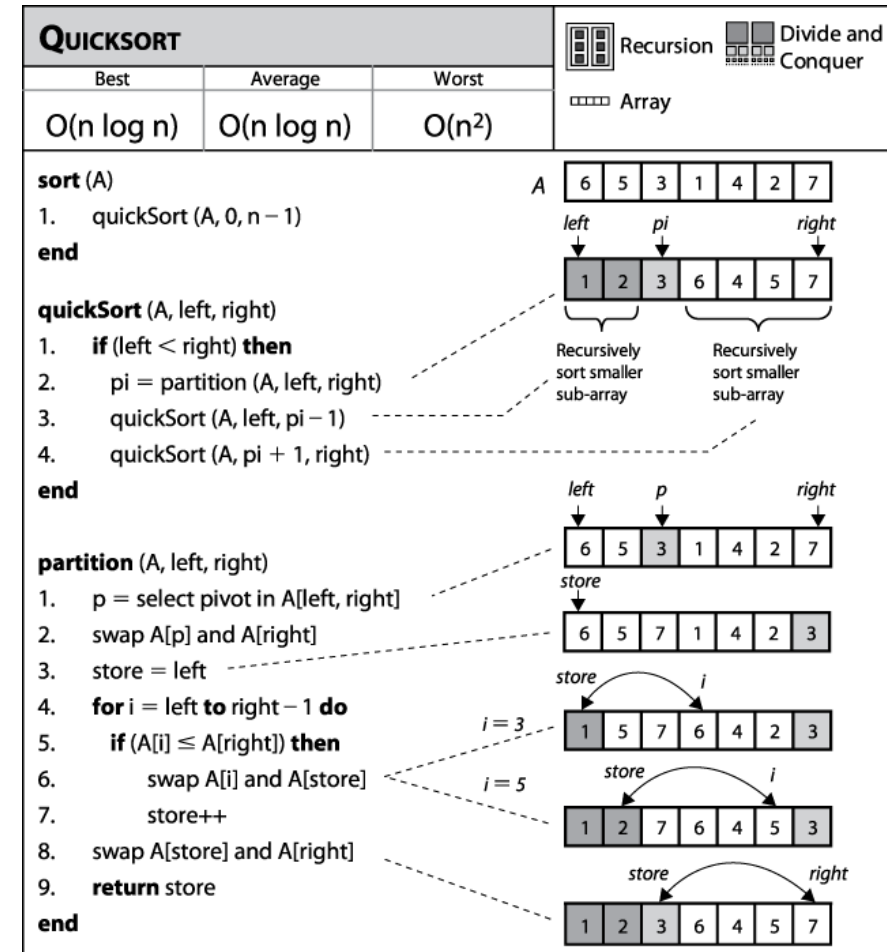
Divide-and-conquer : Quick Sort

Worst case:

1.If pivot is always the smallest/largest, partitioning will produce unbalanced arrays. One array may have $n-1$ elements. Recursion depth will be n and at each level we need to process 'n' elements.

Hence:

$$O(n) \times O(n) = O(n^2)$$



Divide and Conquer : Conclusion

- Used widely in various real world applications across many industries
 - Search Engines: Retrieving data from large databases or indexes, binary search to quickly locate relevant pages
 - Big Data analytics: Break down massive datasets into manageable chunks, process & combine
 - Medical Imaging: Image is divided into smaller parts for analysis, results combined for final diagnostic output
 - E-Commerce and Recommendation Systems: Filtering techniques split the dataset(user data), analyze data in smaller subsets and then recommend.
 - Game development: The game map is divided into smaller nodes/regions and then shortest path is found. (Optimal path)

Greedy algorithm

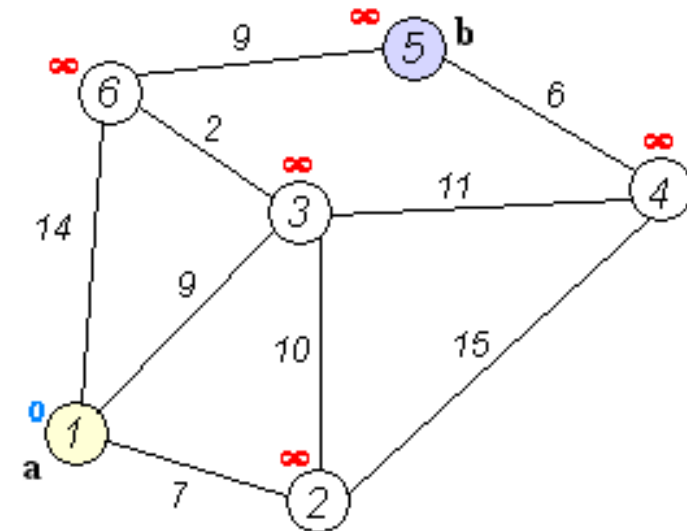
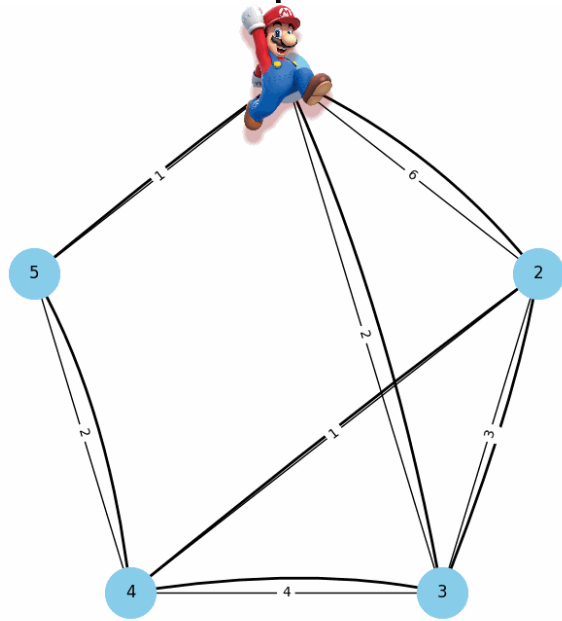
- Optimization problem – Not about just finding a solution but the best solution
- Greedy algorithms work well for optimization problems
- Imagine being at a buffet and you can fill only 1 plate
 - To maximize our satisfaction we pick the most delicious looking food near us without thinking about what might be on the other side of the table.
- Overall steps
 - Make a choice based on current situation that seems best (greedy)
 - Move to next step, keeping the choice fixed hoping that choosing a local optimum at each step, we end up with a global optimum

Greedy Algorithm : Counting Money

- Count a certain amount of money using fewest possible notes or coins
- INR (500,100,20,10,5,1) and we want to go to 639
 - $(1 : 500) + (1 : 100) + (1 : 20) + (1 : 10) + (1 : 5) + (4 : 1) = 9$
- *At each step – take a largest possible note/coin that does not exceed the required.*
- Coins : { 1, 5 , 10 , 25 } , Target : 30
 - Choose 25 (1 coin) + Choose 5 (1 coin) = 2 coins (Optimal)

Greedy Algorithm : Examples

- Dijkstra's Algorithm – Find the shortest path between nodes in a graph. Greedy approach can be used to select smallest known distance from source and proceeds by updating at each point



GPS systems, telecommunication networks, and routing algorithms for packet switching.

Greedy Algorithm : Examples

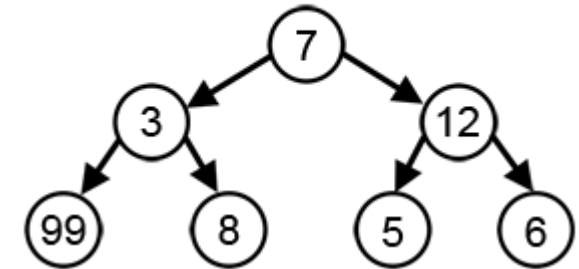
- **Huffman Coding**
 - **Purpose:** Data compression for text, images, and video
 - **Applications:** ZIP files, MP3 audio, JPEG images
 - **Greedy Strategy:** Always merge the two nodes with lowest frequencies
 - **Result:** Frequent characters get shorter codes, rare characters get longer codes
- **Minimum Spanning Tree (Prim's & Kruskal's)**
 - **Purpose:** Connect all nodes with minimum total edge weight
 - **Applications:** Network design (roads, cables, pipelines), circuit layout
 - **Greedy Strategy:**
 - **Prim's:** Always add the cheapest edge to the growing tree
 - **Kruskal's:** Always pick the cheapest edge that doesn't create a cycle

Greedy Algorithm : Examples

- **Activity Selection Problem**
 - **Purpose:** Select maximum number of non-overlapping activities
 - **Applications:** Meeting scheduling, resource allocation, TV programming
 - **Greedy Strategy:** Always pick the activity that finishes earliest
 - **Example:** Scheduling conference rooms to maximize daily booking

Where does it fail ?

- Coins : $\{ 1, 3, 4 \}$, Target : 6
- Greedy approach:
 - Choose 4 (1 coin) + Choose 1 (2 coins) = Total of 3 coins (Non-optimal)
 - Ideally : $3 + 3$
- Coins: $\{ 1, 7, 10 \}$, Target : 15
 - Greedy : $10 + 1 + 1 + 1 + 1 + 1 = 15$ (6 coins)
 - Ideally : $7 + 7 + 1 = 15$ (3 coins)
- Dynamic Programming

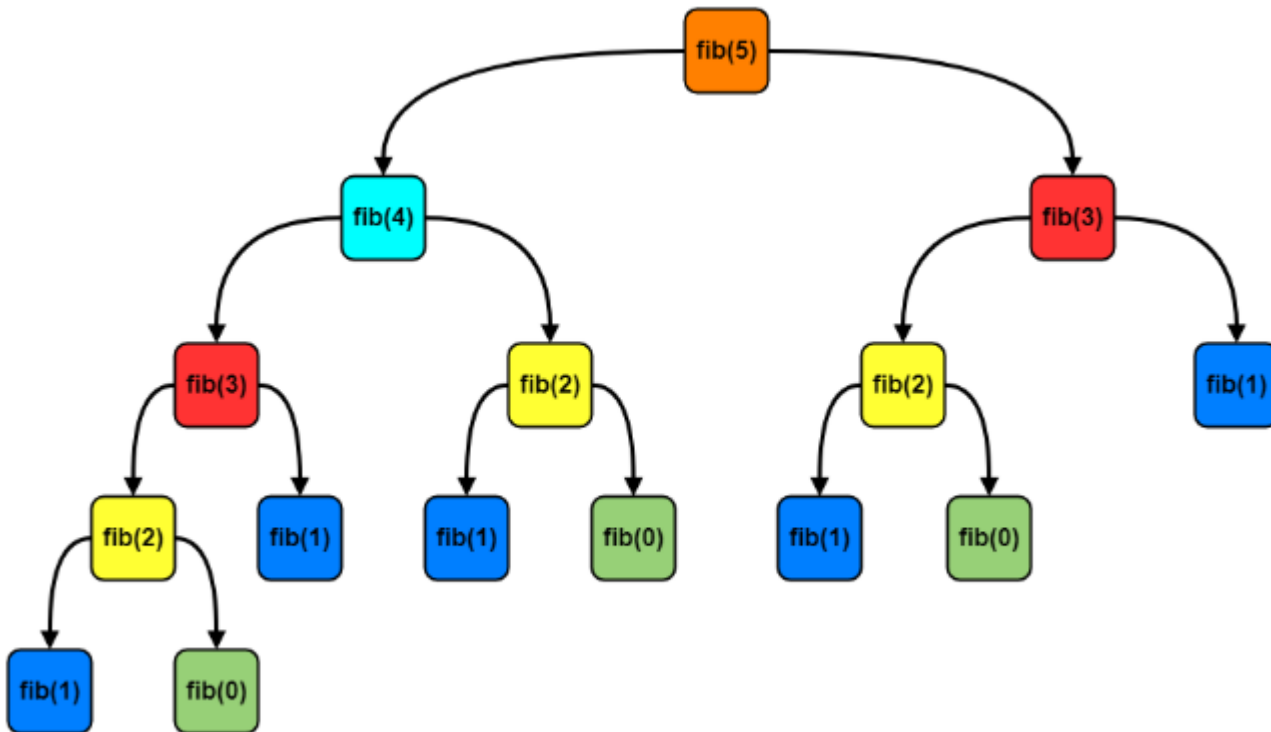


Dynamic Programming

- An algorithm technique
 - Break the problem into **overlapping sub problems**
 - Solve each sub problem **once**
 - **Reuse** the solutions to solve the overall problem
- The essence of dynamic programming is to avoid repeated calculation
- Often, dynamic programming problems are solved using recursion
- Classic example of the same is fibonacci sequence

$$f(n) = f(n - 1) + f(n - 2) \text{ where } n \geq 2 \text{ and } f(0) = 0 \text{ and } f(1) = 1.$$

Calculating Fibonacci using recursion



```
def fibonacci(n):  
    if n == 0 or n == 1:  
        return n  
    else:  
        return fibonacci(n - 1) + fibonacci(n - 2)
```

Time Complexity : $O(2^n)$:

- Each call to fibonacci makes two recursive calls (n-1 and n-2).
- Tree of recursive calls

Optimize using Dynamic Programming

- Time complexity: $O(n)$
- Optimization done using **memoization**(caching already-computed results)
- One of the solutions:

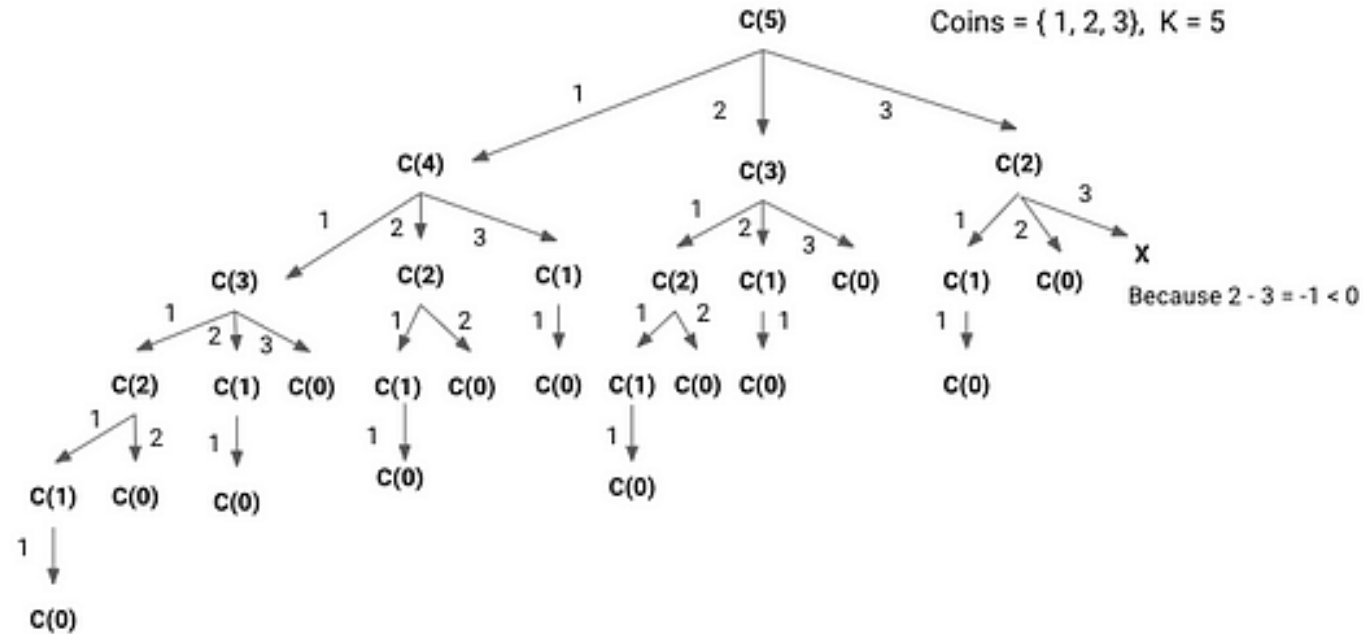
```
memo = {0: 0, 1: 1}

def fibonacci_memoization(n):
    if n in memo.keys():
        return memo[n]
    else:
        memo[n] = fibonacci_memoization(n - 1) + fibonacci_memoization(n - 2)
        return memo[n]
```

Brute force algorithms – Exhaustive search

- Solves a problem by systematically trying all possible solutions until the correct one is found
- Key characteristics
 - Simple and intuitive to implement.
 - Explores all possible options without optimization.
 - Inefficient for large input sizes due to high time complexity.

Coin Change problem using Brute Force



The brute force approach will recursively try every possible way to use the coins to reach the amount

© 2025 Ajay Iyengar. All rights reserved.

Brute Force : Find all substrings

```
public class SubstringExample {  
    public static void main(String[] args) {  
        String str = "abc";  
        System.out.println("All substrings of \"" + str + "\"");  
  
        for (int i = 0; i < str.length(); i++) { // Start position  
            for (int j = i + 1; j <= str.length(); j++) { // End position  
                System.out.println(str.substring(i, j));  
            }  
        }  
    }  
}
```

Time complexity : $O(n^2)$

Brute Force :

Advantages/Disadvantages

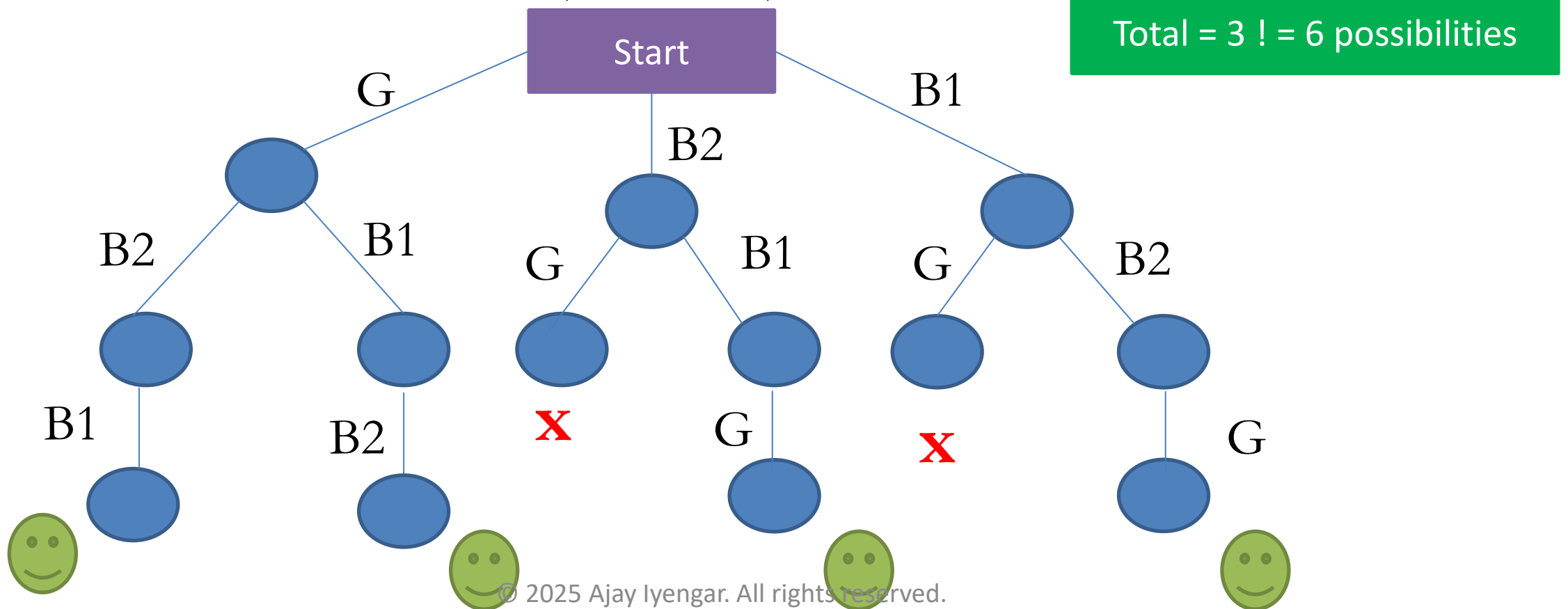
- Advantages
 - Guaranteed to find the correct solution if one exists.
 - Easy to understand and implement.
- Disadvantages
 - Computationally expensive for large inputs.
 - Often impractical for real-world problems with significant data.
- Examples
 - Searching for an element in an unsorted array - Start at the first element and check each elements until target is found
 - Find all permutations of a string – Generate all possible arrangement of the elements
 - Finding the factorial of a number
 - Bubble & Selection sort

Backtracking algorithms

- Another class of algorithms for finding solutions that tries out different possibilities and then **backtracks** when a solution does not work
- Important technique for solving constraint satisfaction problems
- Overall Technique
 - Builds candidates to the solutions step by step
 - Backtracks as soon as it finds that the candidate cannot be completed to a valid solution
- Common problems that can be solved using Backtracking
 - N-Queens, Sudoku, Maze Solving, Travelling Salesman Problem, Word Search Puzzle, Hamiltonian Cycle Problem etc.

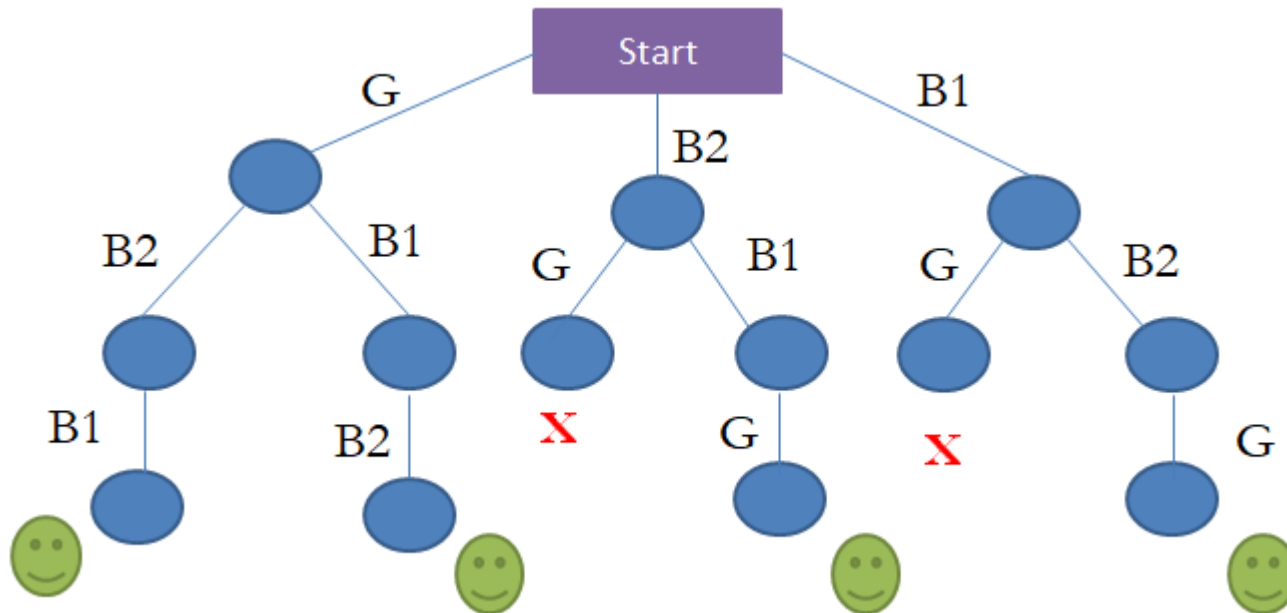
Backtracking : Example 1

- Find possible ways to arrange 2 boys & 1 girl on 3 benches. Girl should not be on the middle bench(constraint)



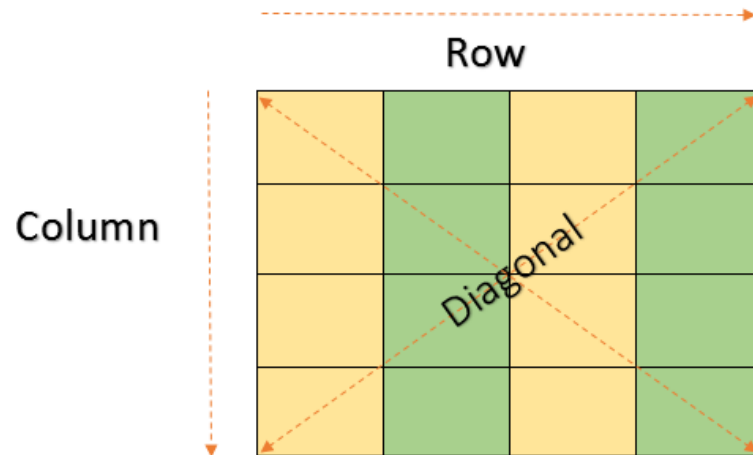
Backtracking: State space tree

- Problems are usually represented as a tree where each node represents a decision and each edge a choice
- Backtracking prunes parts of the search tree



Backtracking: Example 2

- 4 queens problem
 - Problem of placing 4 queens on a 4 *4 chessboard so that no two queens threaten each other. (constraint) - No 2 queens share same row, same column or same diagonal



Solution to the 4 Queen's problem

- 4 queens problem
 - Problem of placing 4 queens on a 4 *4 chessboard so that no two queens threaten each other.

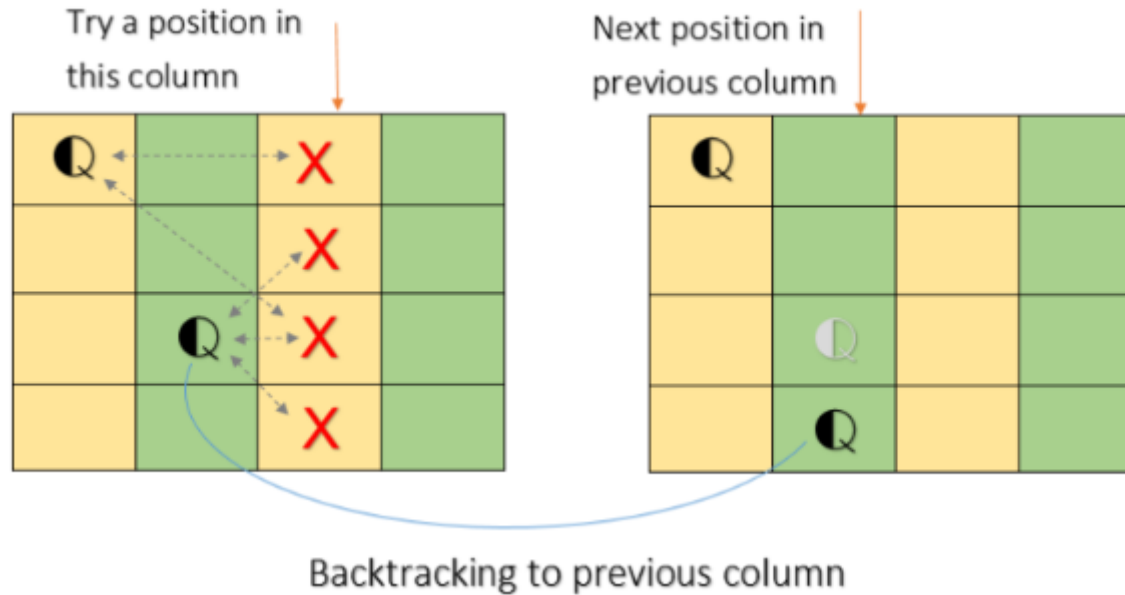
		Q3	
Q1			
			Q4
	Q2		

Solution one

	Q2		
			Q4
Q1			
		Q3	

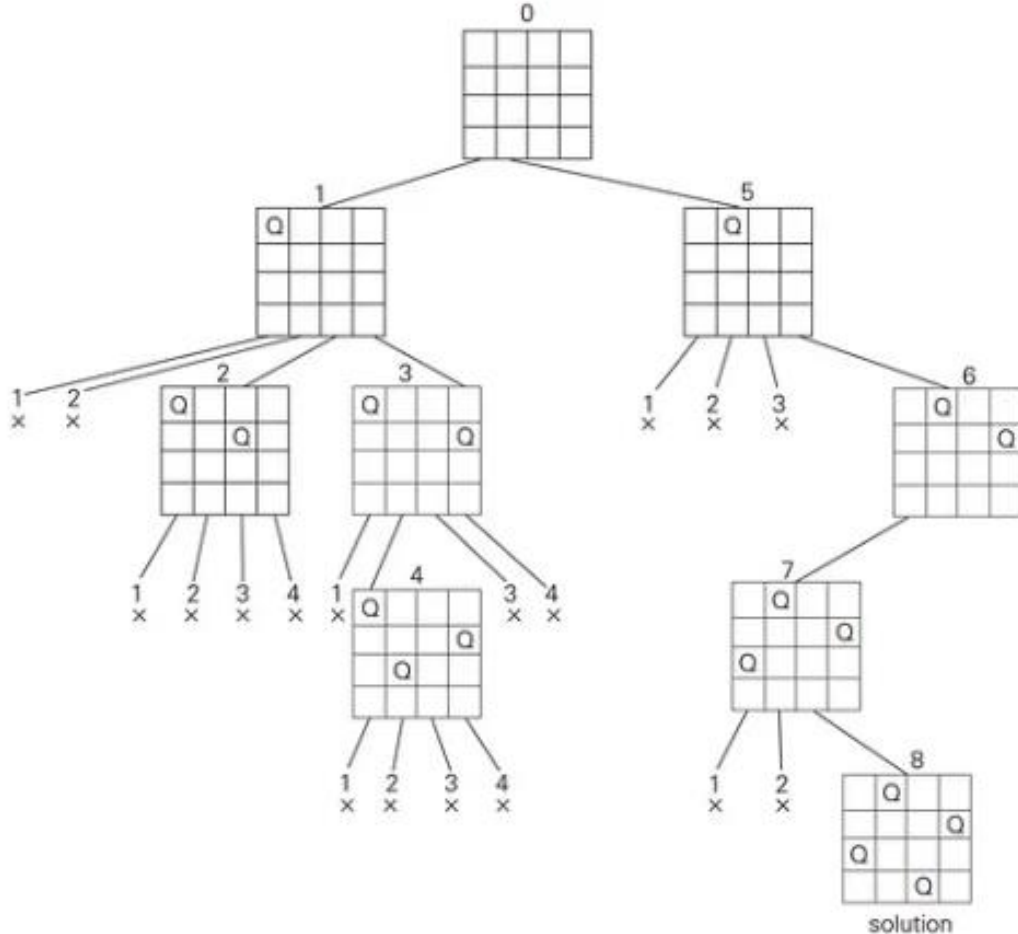
Solution two

Backtracking : Solution to the 4 Queen's problem

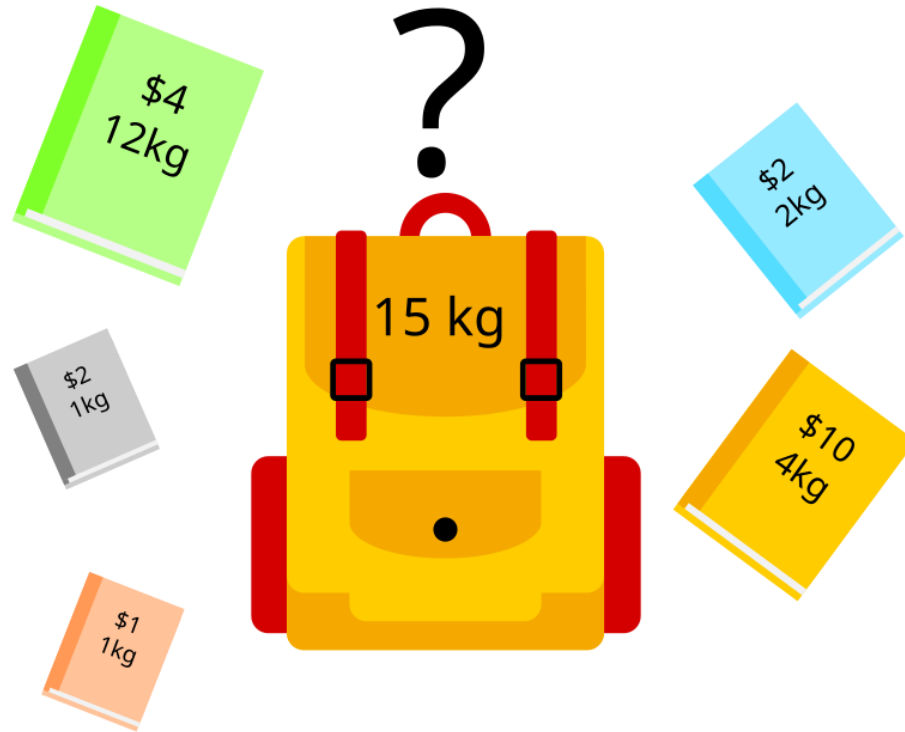


- Solution: <https://ajayiyengar.com/2020/07/27/how-to-solve-the-4-queens-puzzle-in-java/>

Backtracking : State space tree for the 4 Queen's problem (row wise)

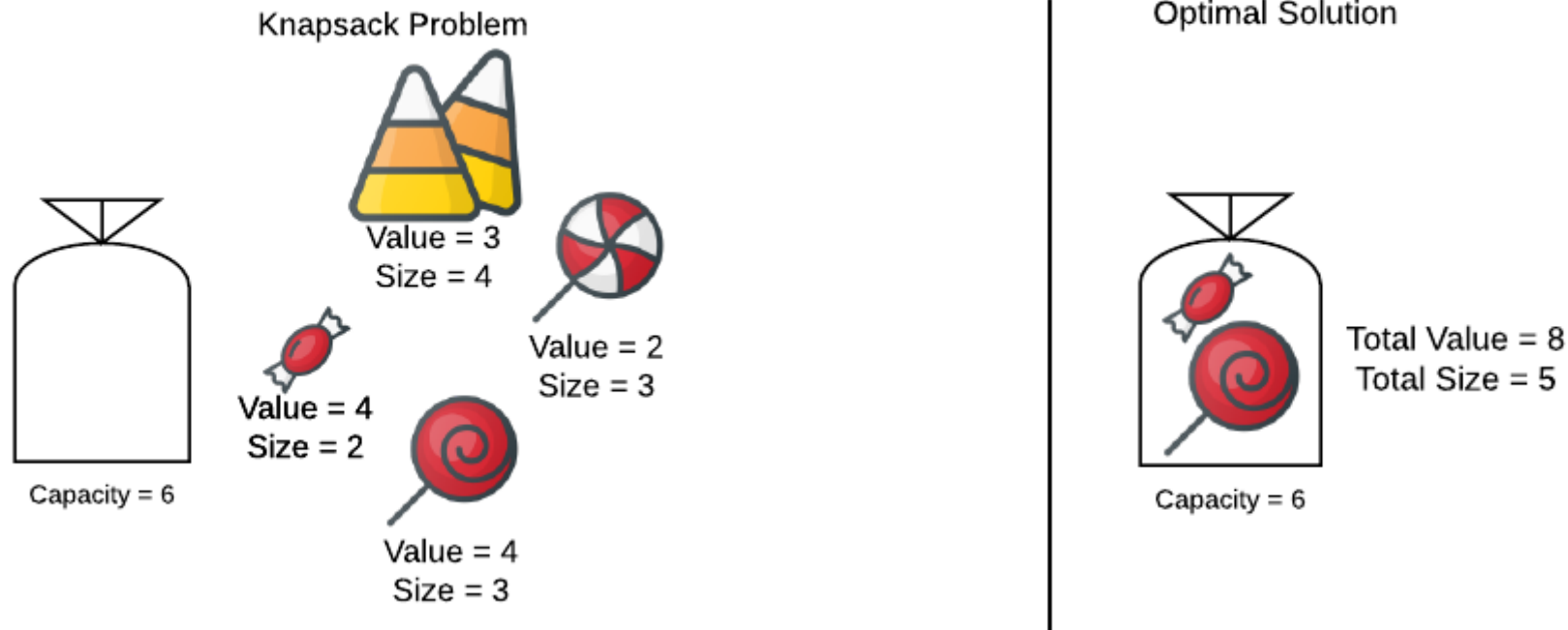


Knapsack problem



Given a set of items, each with a weight and a value, determine which items to include in the collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.

Knapsack problem

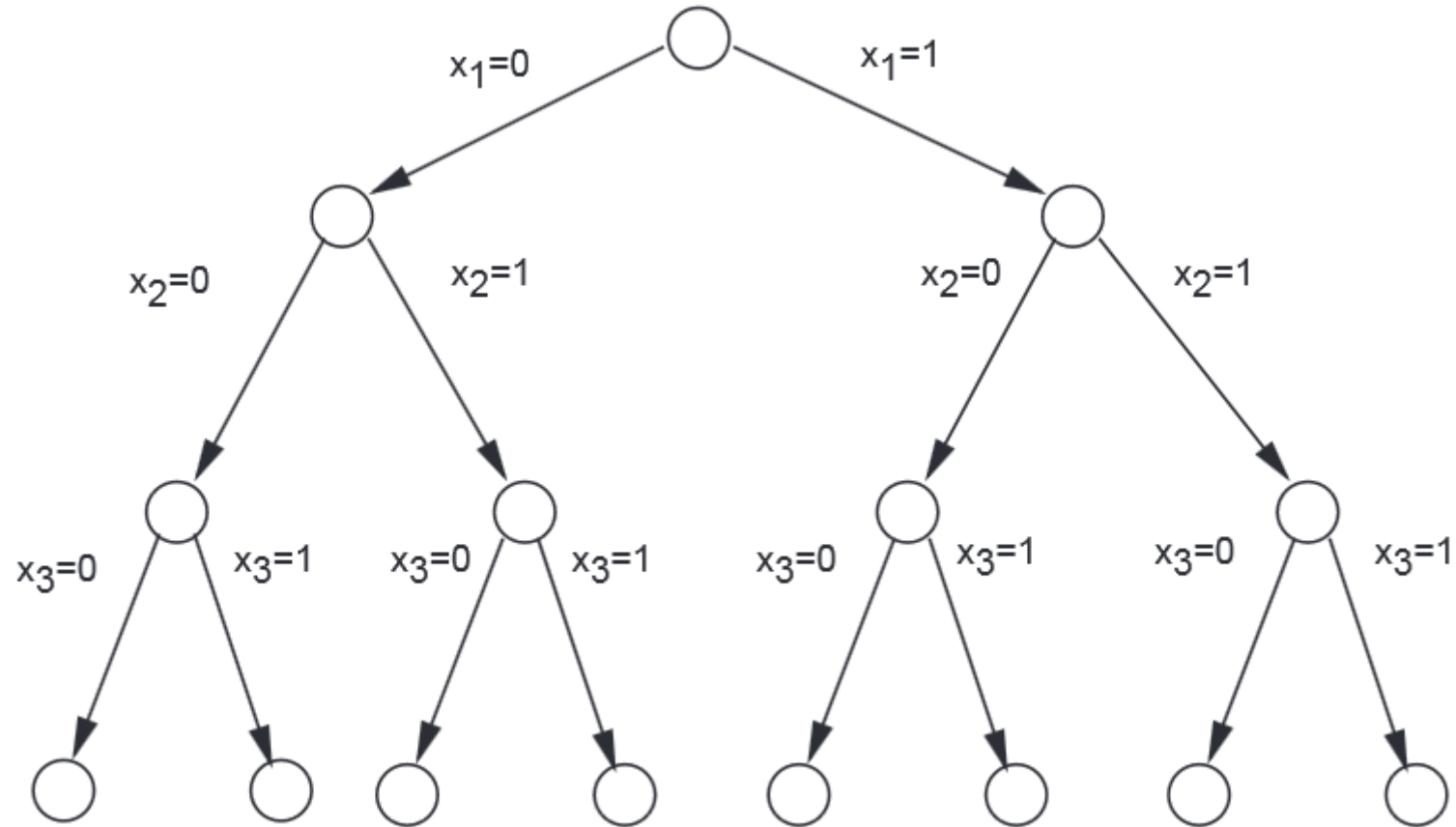


Given a set of items, each with a weight and a value, determine which items to include in the collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.

Branch and Bound algorithms

- Generally used for optimization problems
- As we progress
 - A tree of sub-problems is formed (**Branching**)
 - A method (bounding method) is used to construct a bound (**Bounding**)
 - At each node, apply the bound (**Pruning**)
 - If bounds match – feasible solution for sub-problem
 - If bounds don't match - Discard/Prune that part of the tree
 - Continue, using the best known solutions until all nodes visited

One of the ways of generating solution



Branch and Bound : Knapsack

item	weight	value	$\frac{\text{value}}{\text{weight}}$
1	4	\$40	10
2	7	\$42	6
3	5	\$25	5
4	3	\$12	4

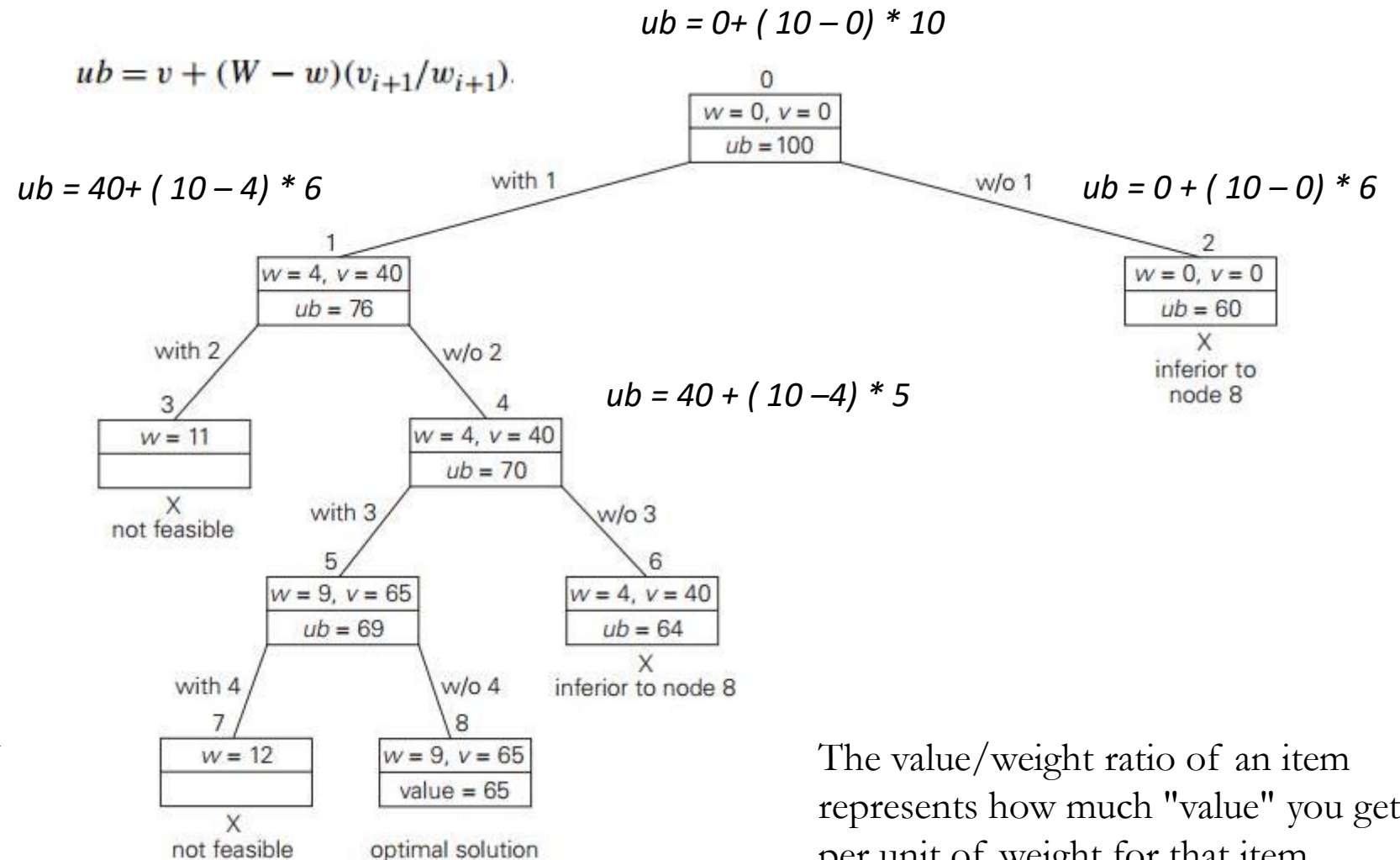
Knapsack capacity $W = 10$

'n' items, here $n = 4$,

$w[i] = \{4, 7, 5, 3\}$,

$v[i] = \{40, 42, 25, 12\}$

Goal : Maximize the total value of items placed in the knapsack without exceeding its weight capacity



The value/weight ratio of an item represents how much "value" you get per unit of weight for that item.

Why do we need the bound ?

- Guides the search
 - Prioritize exploring branches that might contain the optimal solution
- Prune non-promising branches
 - If upper bound is $<$ current best solution, then this branch cannot lead to a better solution
- Without bound, we will have to explore all branches

Branch and Bound : Key Components

- **Branching:**
 - Divide the problem into smaller sub problems.
 - Represent these sub problems as branches of a tree.
- **Bounding:**
 - For each sub problem, calculate a bound (an estimate of the best solution that can be achieved from that sub problem).
 - This helps decide whether to explore a branch further or prune it.
- **Pruning:**
 - If a branch's bound shows that it cannot improve on the current best solution, it is pruned (discarded).

Branch and Bound : Conclusion

- Guarantees an optimal solution when the bounding function is well-designed.
- Can be memory intensive
- Applications
 - Resource allocation and selection problems
 - Job scheduling : Optimize tasks or resource usage
 - Travelling Salesman problem/Vehicle Routing Problem - Path optimization in logistics
 - Financial Applications – Portfolio Optimization
 - AI & Machine Learning